CHAPTER

TWENTYONE

PROTOCOLS — STRING SERVICES

21.1 Unicode Collation Protocol

This section defines the Unicode Collation protocol. This protocol is used to allow code running in the boot services environment to perform lexical comparison functions on Unicode strings for given languages.

21.1.1 EFI_UNICODE_COLLATION_PROTOCOL

Summary

Is used to perform case-insensitive comparisons of strings.

GUID

```
#define EFI_UNICODE_COLLATION_PROTOCOL2_GUID \
  {0xa4c751fc, 0x23ae, 0x4c3e, \
    {0x92, 0xe9, 0x49, 0x64, 0xcf, 0x63, 0xf3, 0x49}}
```

Protocol Interface Structure

```
typedef struct {
  EFI_UNICODE_COLLATION_STRICOLL         StriColl;
  EFI_UNICODE_COLLATION_METAIMATCH       MetaiMatch;
  EFI_UNICODE_COLLATION_STRLWR           StrLwr;
  EFI_UNICODE_COLLATION_STRUPR           StrUpr;
  EFI_UNICODE_COLLATION_FATTOSTR         FatToStr;
  EFI_UNICODE_COLLATION_STRTOFAT         StrToFat;
  CHAR8                                  *SupportedLanguages;
} EFI_UNICODE_COLLATION_PROTOCOL;
```

Parameters

StriColl
Performs a case-insensitive comparison of two Null-terminated strings. See the *EFI_UNICODE_COLLATION_PROTOCOL.StriColl()* function description.

MetaiMatch
Performs a case-insensitive comparison between a Null-terminated pattern string and a Null-terminated string. The pattern string can use the '?' wildcard to match any character, and the '*' wildcard to match any substring. See the *EFI_UNICODE_COLLATION_PROTOCOL.MetaiMatch()* function description.

StrLwr
Converts all the characters in a Null-terminated string to lowercase characters. See the *EFI_UNICODE_COLLATION_PROTOCOL.StrLwr()* function description.

StrUpr
Converts all the characters in a Null-terminated string to uppercase characters. See the *EFI_UNICODE_COLLATION_PROTOCOL.StrUpr()* function description.

FatToStr
Converts an 8.3 FAT file name using an OEM character set to a Null-terminated string. See the *EFI_UNICODE_COLLATION_PROTOCOL.FatToStr()* function description.

StrToFat
Converts a Null-terminated string to legal characters in a FAT filename using an OEM character set. See the *EFI_UNICODE_COLLATION_PROTOCOL.StrToFat()* function description.

SupportedLanguages
A Null-terminated ASCII string array that contains one or more language codes. This array is specified in RFC 4646 format. See *Formats — Language Codes and Language Code Arrays*

Description

The EFI_UNICODE_COLLATION_PROTOCOL is used to perform case-insensitive comparisons of strings.

One or more of the EFI_UNICODE_COLLATION_PROTOCOL instances may be present at one time. Each protocol instance can support one or more language codes. The language codes supported in the EFI_UNICODE_COLLATION_PROTOCOL are declared in *SupportedLanguages* .

The *SupportedLanguages* is a Null-terminated ASCII string array that contains one or more supported language codes. This is the list of language codes that this protocol supports. See *Formats — Language Codes and Language Code Arrays* for the format of language codes and language code arrays.

The main motivation for this protocol is to help support file names in a file system driver. When a file is opened, a file name needs to be compared to the file names on the disk. In some cases, this comparison needs to be performed in a case-insensitive manner. In addition, this protocol can be used to sort files from a directory or to perform a case-insensitive file search.

21.1.2 EFI_UNICODE_COLLATION_PROTOCOL.StriColl()

Summary

Performs a case-insensitive comparison of two Null-terminated strings.

Prototype

```
typedef
INTN
(EFIAPI *EFI_UNICODE_COLLATION_STRICOLL) (
  IN EFI_UNICODE_COLLATION_PROTOCOL    *This,
  IN CHAR16                            *s1,
  IN CHAR16                            *s2
  );
```

Parameters

This
A pointer to the *EFI_UNICODE_COLLATION_PROTOCOL* instance. Type EFI_UNICODE_COLLATION_PROTOCOL is defined above.

s1
A pointer to a Null-terminated string.

s2
A pointer to a Null-terminated string.

Description

The StriColl() function performs a case-insensitive comparison of two Null-terminated strings.

This function performs a case-insensitive comparison between the string s1 and the string s2 using the rules for the language codes that this protocol instance supports. If s1 is equivalent to s2, then 0 is returned. If s1 is lexically less than s2, then a negative number will be returned. If s1 is lexically greater than s2, then a positive number will be returned. This function allows strings to be compared and sorted.

Status Codes Returned

0	s1 is equivalent to s2.
> 0	s1 is lexically greater than s2.
< 0	s1 is lexically less than s2.

21.1.3 EFI_UNICODE_COLLATION_PROTOCOL.MetaiMatch()

Summary

Performs a case-insensitive comparison of a Null-terminated pattern string and a Null-terminated string.

Prototype

```
typedef
BOOLEAN
(EFIAPI *EFI_UNICODE_COLLATION_METAIMATCH) (
  IN EFI_UNICODE_COLLATION_PROTOCOL       *This,
  IN CHAR16                               *String,
  IN CHAR16                               *Pattern
  );
```

Parameters

This
A pointer to the *EFI_UNICODE_COLLATION_PROTOCOL* instance. Type EFI_UNICODE_COLLATION_PROTOCOL is defined above.

String
A pointer to a Null-terminated string.

Pattern
A pointer to a Null-terminated string.

Description

The *MetaiMatch()* function performs a case-insensitive comparison of a Null-terminated pattern string and a Null-terminated string.

This function checks to see if the pattern of characters described by *Pattern* are found in *String* . The pattern check is a case-insensitive comparison using the rules for the language codes that this protocol instance supports. If the pattern match succeeds, then **TRUE** is returned. Otherwise **FALSE** is returned. The following syntax can be used to build the string Pattern:

```
*                              Match 0 or more characters.
?                              Match any one character.
[<char1><char2>...<charN>]     Match any character in the set.
[<char1>-<char2>]              Match any character between <char1> and<char2>.
<char>                         Match the character <char>.
```

Following is an example pattern for English:

```
*.FW                           Matches all strings that end in ".FW" or .fw" or ".Fw" or
↪".fW."
[a-z]                          Match any letter in the alphabet.
[!@#$%^&*()]                   Match any one of these symbols.
z                              Match the character "z" or "Z."
D?.*                           Match the character "D" or "d"
                               followed by any character
                               followed by a "." followed by any string.
```

Status Codes Returned

TRUE	Pattern was found in String.
FALSE	Pattern was not found in String.

21.1.4 EFI_UNICODE_COLLATION_PROTOCOL.StrLwr()

Summary

Converts all the characters in a Null-terminated string to lowercase characters.

Prototype

```
typedef
VOID
(EFIAPI *EFI_UNICODE_COLLATION_STRLWR) (
  IN EFI_UNICODE_COLLATION_PROTOCOL       *This,
  IN OUT CHAR16                           *String
  );
```

Parameters

This
A pointer to the *EFI_UNICODE_COLLATION_PROTOCOL* instance. Type EFI_UNICODE_COLLATION_PROTOCOL is defined above.

String
A pointer to a Null-terminated string.

Description

This function walks through all the characters in *String,* and converts each one to its lowercase equivalent if it has one. The converted string is returned in *String* .

21.1.5 EFI_UNICODE_COLLATION_PROTOCOL.StrUpr()

Summary

Converts all the characters in a Null-terminated string to uppercase characters.

Prototype

```
typedef
VOID
(EFIAPI *EFI_UNICODE_COLLATION_STRUPR) (
  IN EFI_UNICODE_COLLATION_PROTOCOL      *This,
  IN OUT CHAR16                          *String
  );
```

Parameters

This
A pointer to the *EFI_UNICODE_COLLATION_PROTOCOL* instance. Type EFI_UNICODE_COLLATION_PROTOCOL is defined above.

String
A pointer to a Null-terminated string.

Description

This functions walks through all the characters in *String,* and converts each one to its uppercase equivalent if it has one. The converted string is returned in *String* .

21.1.6 EFI_UNICODE_COLLATION_PROTOCOL.FatToStr()

Summary

Converts an 8.3 FAT file name in an OEM character set to a Null-terminated string.

Prototype

```
typedef
VOID
(EFIAPI *EFI_UNICODE_COLLATION_FATTOSTR) (
  IN EFI_UNICODE_COLLATION_PROTOCOL       *This,
  IN UINTN                                FatSize,
  IN CHAR8                                *Fat,
  OUT CHAR16                              *String
  );
```

Parameters

This
A pointer to the *EFI_UNICODE_COLLATION_PROTOCOL* instance. Type EFI_UNICODE_COLLATION_PROTOCOL is defined above.

FatSize
The size of the string Fat in bytes.

Fat
A pointer to a Null-terminated string that contains an 8.3 file name encoded using an 8-bit OEM character set.

String
A pointer to a Null-terminated string. The string must be allocated in advance to hold *FatSize* characters.

Description

This function converts the string specified by *Fat* with length *FatSize* to the Null-terminated string specified by *String* . The characters in *Fat* are from an OEM character set.

21.1.7 EFI_UNICODE_COLLATION_PROTOCOL.StrToFat()

Summary

Converts a Null-terminated string to legal characters in a FAT filename using an OEM character set.

Prototype

```
typedef
BOOLEAN
(EFIAPI *EFI_UNICODE_COLLATION_STRTOFAT) (
  IN EFI_UNICODE_COLLATION_PROTOCOL       *This,
  IN CHAR16                               *String,
  IN UINTN                                FatSize,
  OUT CHAR8                               *Fat
  );
```

Parameters

This
A pointer to the *EFI_UNICODE_COLLATION_PROTOCOL* instance. Type EFI_UNICODE_COLLATION_PROTOCOL is defined above.

String
A pointer to a Null-terminated string.

FatSize
The size of the string Fat in bytes.

Fat
A pointer to a string that contains the converted version of *String* using legal FAT characters from an OEM character set.

Description

This function converts the characters from *String* into legal FAT characters in an OEM character set and stores then in the string *Fat* . This conversion continues until either *FatSize* bytes are stored in *Fat,* or the end of *String* is reached. The characters '.' (period) and ' ' (space) are ignored for this conversion. Characters that map to an illegal FAT character are substituted with an '_'. If no valid mapping from a character to an OEM character is available, then it is also substituted with an '_'. If any of the character conversions are substituted with a '_', then **TRUE** is returned. Otherwise **FALSE** is returned.

Status Codes Returned

TRUE	One or more conversions failed and were substituted with '_'.
FALSE	None of the conversions failed.

21.2 Regular Expression Protocol

This section defines the Regular Expression Protocol. This protocol is used to match Unicode strings against Regular Expression patterns.

21.2.1 EFI_REGULAR_EXPRESSION_PROTOCOL

Summary

GUID

```
#define EFI_REGULAR_EXPRESSION_PROTOCOL_GUID \
  { 0xB3F79D9A, 0x436C, 0xDC11,\
    { 0xB0, 0x52, 0xCD, 0x85, 0xDF, 0x52, 0x4C, 0xE6 } }
```

Protocol Interface Structure

```
typedef struct {
  EFI_REGULAR_EXPRESSION_MATCH      MatchString;
  EFI_REGULAR_EXPRESSION_GET_INFO   GetInfo;
} EFI_REGULAR_EXPRESSION_PROTOCOL;
```

Parameters

MatchString
Search the input string for anything that matches the regular expression.

GetInfo
Returns information about the regular expression syntax types supported by the implementation.

21.2.2 EFI_REGULAR_EXPRESSION_PROTOCOL.MatchString()

Summary

Checks if the input string matches to the regular expression pattern.

Prototype

```
typedef
EFI_STATUS
EFIAPI *EFI_REGULAR_EXPRESSION_MATCH) (
  IN   EFI_REGULAR_EXPRESSION_PROTOCOL *This,
  IN   CHAR16                          *String,
  IN   CHAR16                          *Pattern,
  IN   EFI_REGEX_SYNTAX_TYPE           *SyntaxType, OPTIONAL
  OUT  BOOLEAN                         *Result,
  OUT  EFI_REGEX_CAPTURE               **Captures, OPTIONAL
  OUT UINTN                            *CapturesCount
  );
```

Parameters

This
A pointer to the *EFI_REGULAR_EXPRESSION_PROTOCOL* instance. Type EFI_REGULAR_EXPRESSION_PROTOCOL is defined in above.

String
A pointer to a NULL terminated string to match against the regular expression string specified by *Pattern* .

Pattern
A pointer to a NULL terminated string that represents the regular expression.

SyntaxType
A pointer to the EFI_REGEX_SYNTAX_TYPE that identifies the regular expression syntax type to use. May be NULL in which case the function will use its default regular expression syntax type.

Result
On return, points to TRUE if String fully matches against the regular expression Pattern using the regular expression SyntaxType . Otherwise, points to FALSE .

Captures
A Pointer to an array of EFI_REGEX_CAPTURE objects to receive the captured groups in the event of a match. The full sub-string match is put in Captures [0], and the results of N capturing groups are put in Captures [1:N]. If Captures is NULL, then this function doesn't allocate the memory for the array and does not build up the elements. It only returns the number of matching patterns in CapturesCount . If Captures is not NULL, this function returns a pointer to an array and builds up the elements in the array. CapturesCount is also updated to the number of matching patterns found. It is the caller's responsibility to free the memory pool in Captures and in each CapturePtr in the array elements.

CapturesCount
On output, CapturesCount is the number of matching patterns found in String. Zero means no matching patterns were found in the string.

Description

The MatchString() function performs a matching of a Null-terminated input string with the NULL terminated pattern string. The pattern string syntax type is optionally identified in SyntaxType .

This function checks to see if String fully matches against the regular expression described by Pattern. The pattern check is performed using regular expression rules that are supported by this implementation, as indicated in the return value of GetInfo function. If the pattern match succeeds, then TRUE is returned in Result . Otherwise FALSE is returned.

Related Definitions

```
typedef struct {
  CONST CHAR16       *CapturePtr;
  UINTN              Length;
}  EFI_REGEX_CAPTURE;
```

***CapturePtr**
Pointer to the start of the captured sub-expression within matched String.

Length
Length of captured sub-expression.

Status Codes Returned

EFI_SUCCESS	The regular expression string matching completed successfully.
EFI_UNSUPPORTED	The regular expression syntax specified by *SyntaxType* is not supported by this driver.
EFI_DEVICE_ERROR	The regular expression string matching failed due to a hardware or firmware error.
EFI_INVALID_PARAMETER	String, Pattern, Result, or CapturesCount is NULL.

21.2.3 EFI_REGULAR_EXPRESSION_PROTOCOL.GetInfo()

Summary

Returns information about the regular expression syntax types supported by the implementation.

Prototype

```
typedef
EFI_STATUS
EFIAPI *EFI_REGULAR_EXPRESSION_GET_INFO) (
  IN EFI_REGULAR_EXPRESSION_PROTOCOL         *This,
  IN OUT UINTN                               *RegExSyntaxTypeListSize,
  OUT EFI_REGEX_SYNTAX_TYPE                  *RegExSyntaxTypeList
  );
```

Parameters

This
A pointer to the *EFI_REGULAR_EXPRESSION_PROTOCOL* instance.

RegExSyntaxTypeListSize
On input, the size in bytes of *RegExSyntaxTypeList* . *On* output with a return code of *EFI_SUCCESS* , *th* e size in bytes of the data returned in *RegExSyntaxTypeList* . On output with a return code of *EFI_BUFFER_TOO_SMALL,* the size of *RegExSyntaxTypeList* required to obtain the list.

RegExSyntaxTypeList
A caller-allocated memory buffer filled by the driver with one *EFI_REGEX_SYNTAX_TYPE* element for each supported regular expression syntax type. The list must not change across multiple calls to the same driver. The first syntax type in the list is the default type for the driver.

Description

This function returns information about supported regular expression syntax types. A driver implementing the *EFI_REGULAR_EXPRESSION_PROTOCOL* need not support more than one regular expression syntax type, but shall support a minimum of one regular expression syntax type.

Related Definitions

```
typedef EFI_GUID EFI_REGEX_SYNTAX_TYPE;
```

Status Codes Returned

EFI_SUCCESS	The regular expression syntax types list was returned successfully.
EFI_UNSUPPORTED	The service is not supported by this driver.
EFI_DEVICE_ERROR	The list of syntax types could not be retrieved due to a hardware or firmware error.
EFI_BUFFER_TOO_SMALL	The buffer *RegExSyntaxTypeList* is too small to hold the result.
EFI_INVALID_PARAMETER	*RegExSyntaxTypeListSize* is NULL.

21.2.4 EFI Regular Expression Syntax Type Definitions

Summary

This sub-section provides EFI_GUID values for a selection of EFI_REGULAR_EXPRESSION_PROTOCOL syntax types. The types listed are optional, not meant to be exhaustive and may be augmented by vendors or other industry standards.

Prototype

For regular expression rules specified in the POSIX Extended Regular Expression (ERE) Syntax:

```
#define EFI_REGEX_SYNTAX_TYPE_POSIX_EXTENDED_GUID \
  {0x5F05B20F, 0x4A56, 0xC231,\
    { 0xFA, 0x0B, 0xA7, 0xB1, 0xF1, 0x10, 0x04, 0x1D }}
```

For regular expression rules specified in the Perl standard:

```
#define EFI_REGEX_SYNTAX_TYPE_PERL_GUID \
  {0x63E60A51, 0x497D, 0xD427,\
   { 0xC4, 0xA5, 0xB8, 0xAB, 0xDC, 0x3A, 0xAE, 0xB6 }}
```

For regular expression rules specified in the ECMA 262 Specification:

```
#define EFI_REGEX_SYNTAX_TYPE_ECMA_262_GUID \
  { 0x9A473A4A, 0x4CEB, 0xB95A, 0x41,\
    { 0x5E, 0x5B, 0xA0, 0xBC, 0x63, 0x9B, 0x2E }}
```

For regular expression rules specified in the POSIX Extended Regular Expression (ERE) Syntax, where the Pattern and String input strings need to be converted to ASCII:

```
#define EFI_REGEX_SYNTAX_TYPE_POSIX_EXTENDED_ASCII_GUID \
  {0x3FD32128, 0x4BB1, 0xF632, \
    { 0xBE, 0x4F, 0xBA, 0xBF, 0x85, 0xC9, 0x36, 0x76 }}
```

For regular expression rules specified in the Perl standard, where the Pattern and String input strings nees to be converted to ASCII:

```
#define EFI_REGEX_SYNTAX_TYPE_PERL_ASCII_GUID \
  {0x87DFB76D, 0x4B58, 0xEF3A, \
    { 0xF7, 0xC6, 0x16, 0xA4, 0x2A, 0x68, 0x28, 0x10 }}
```

For regular expression rules specified in the ECMA 262 Specification, where the Pattern and String input strings need to be converted to ASCII:

```
#define EFI_REGEX_SYNTAX_TYPE_ECMA_262_ASCII_GUID \
  { 0xB2284A2F, 0x4491, 0x6D9D, \
    { 0xEA, 0xB7, 0x11, 0xB0, 0x67, 0xD4, 0x9B, 0x9A }}
```

See *References* for more information.